llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5 · claude-haiku-4-5-20251001 · BROKEN · RIGHT = attempt 4 of 80, rejected by the gate at 'tsan'

−5 / +108 lines of 1967 · 6 parallel region(s) added · 26 of 80 candidate(s) kept · RESTRUCTURING (81 code line(s) added, 5 removed) · restructuring level 0

BEFORE

```
1953  {
1954  #ifdef SEDOV_SYNC_POS_VEL_LATE
1955     Domain_member fieldData[6] ;
1956  #endif
1957
1958     LagrangeNodal(domain);
1959
1960  #ifdef SEDOV_SYNC_POS_VEL_LATE
1961  #endif
1962
1963     LagrangeElements(domain, domain.numElem());
1964
1965     CalcTimeConstraintsForElems(domain);
```

ATTEMPT 4 OF 80, REJECTED BY THE GATE AT 'TSAN'

file

```
1953  {
1954  #ifdef SEDOV_SYNC_POS_VEL_LATE
1955     Domain_member fieldData[6] ;
1956  #endif
1957
1958     // Inline and parallelize all stages into one large parallel region to amor
1959     // Stage 1: Calculate forces on nodes (sequential, complex accumulation)
1960     {
1961        Index_t numNode = domain.numNode();
1962        for (Index_t i = 0; i < numNode; ++i) {
1963           domain.fx(i) = Real_t(0.0);
1964           domain.fy(i) = Real_t(0.0);
1965           domain.fz(i) = Real_t(0.0);
1966        }
1967        CalcVolumeForceForElems(domain);
1968     }
1969
1970     // Stage 2: Kinematics on elements (sequential, complex calculations)
1971     CalcLagrangeElements(domain);
1972
1973     const Real_t delt = domain.deltatime();
1974     Real_t u_cut = domain.u_cut();
1975     Index_t numNode = domain.numNode();
1976     Real_t qqc = domain.qqc();
1977     Real_t dvovmax = domain.dvovmax();
1978     Real_t qqc2 = Real_t(64.0) * qqc * qqc;
1979     Real_t dtcourant_local = 1.0e+20;
1980     Real_t dthydro_local = 1.0e+20;
1981
1982     // Stages 3-7: Node updates (acceleration, velocity, position) and element/
1983     // Parallelize with one large region over nodes and elements
1984  #pragma omp parallel
1985     {
1986        // Stage 3: Acceleration and velocity update on all nodes
1987  #pragma omp for schedule(static)
1988        for (Index_t i = 0; i < numNode; ++i) {
1989           domain.xdd(i) = domain.fx(i) / domain.nodalMass(i);
1990           domain.ydd(i) = domain.fy(i) / domain.nodalMass(i);
1991           domain.zdd(i) = domain.fz(i) / domain.nodalMass(i);
1992
1993           Real_t xdtmp = domain.xd(i) + domain.xdd(i) * delt;
1994           if (FABS(xdtmp) < u_cut) xdtmp = Real_t(0.0);
1995           domain.xd(i) = xdtmp;
1996
1997           Real_t ydtmp = domain.yd(i) + domain.ydd(i) * delt;
1998           if (FABS(ydtmp) < u_cut) ydtmp = Real_t(0.0);
1999           domain.yd(i) = ydtmp;
2000
2001           Real_t zdtmp = domain.zd(i) + domain.zdd(i) * delt;
2002           if (FABS(zdtmp) < u_cut) zdtmp = Real_t(0.0);
2003           domain.zd(i) = zdtmp;
2004
2005           domain.x(i) += domain.xd(i) * delt;
2006           domain.y(i) += domain.yd(i) * delt;
2007           domain.z(i) += domain.zd(i) * delt;
2008        }
2009
2010  #pragma omp single
2011        {
2012           ApplyAccelerationBoundaryConditionsForNodes(domain);
2013        }
2014
2015        // Stage 4: Apply material properties (regions with complex inner loops)
2016  #pragma omp for schedule(dynamic)
2017        for (Index_t i = 0; i < domain.numElem(); ++i) {
2018           Real_t vdov = domain.dxx(i) + domain.dyy(i) + domain.dzz(i);
2019           Real_t vdovthird = vdov / Real_t(3.0);
2020           domain.vdov(i) = vdov;
2021           domain.dxx(i) -= vdovthird;
2022           domain.dyy(i) -= vdovthird;
2023           domain.dzz(i) -= vdovthird;
2024        }
2025
2026        // Stage 5: Time constraints over all elements
2027  #pragma omp for reduction(min:dtcourant_local, dthydro_local) schedule(dynamic
2028        for (Index_t k = 0; k < domain.numElem(); ++k) {
2029           // Courant constraint
2030           if (domain.vdov(k) != Real_t(0.)) {
2031              Real_t dtf = domain.ss(k) * domain.ss(k);
2032              if (domain.vdov(k) < Real_t(0.)) {
2033                 dtf = dtf + qqc2 * domain.arealg(k) * domain.arealg(k) *
2034                       domain.vdov(k) * domain.vdov(k);
2035              }
2036              dtf = SQRT(dtf);
2037              dtf = domain.arealg(k) / dtf;
2038              if (dtf < dtcourant_local) {
2039                 dtcourant_local = dtf;
2040              }
2041           }
2042
2043           // Hydro constraint
2044           if (domain.vdov(k) != Real_t(0.)) {
2045              Real_t dtdvov = dvovmax / (FABS(domain.vdov(k)) + Real_t(1.e-20));
2046              if (dtdvov < dthydro_local) {
2047                 dthydro_local = dtdvov;
```

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is